

Thuật toán dòng chảy mạng (Network Flow Algorithms)

Giảng viên: Trần Đình Tân

Môn: Thuật toán ứng dụng

Ngày 2 tháng 7 năm 2026

Sau bài học, người học có thể:

- Mô hình hóa được một bài toán bằng mạng dòng chảy có nguồn, đích và dung lượng.
- Phát biểu đúng các ràng buộc của một luồng hợp lệ: ràng buộc dung lượng và bảo toàn luồng.
- Xây dựng được mạng dư, xác định đường tăng luồng và nút cổ chai.
- Trình bày được thuật toán Ford–Fulkerson và biến thể Edmonds–Karp.
- Giải thích được định lý max-flow min-cut và dùng lát cắt để kiểm chứng tối ưu.
- Áp dụng mô hình dòng chảy cho ghép cặp hai phía và bài toán phân công có sức chứa.

Nội dung

- 1 Mô hình dòng chảy
- 2 Mạng dư và đường tăng luồng
- 3 Ford–Fulkerson và Edmonds–Karp
- 4 Ví dụ chạy tay
- 5 Ứng dụng mô hình hóa
- 6 Cài đặt và kiểm thử
- 7 Tổng kết và bài tập

Vì sao cần mô hình dòng chảy?

Ý chính

Dòng chảy mạng là mô hình thống nhất cho nhiều bài toán phân phối tài nguyên hữu hạn.

- Ví dụ vận tải: hàng hóa đi từ kho đến các điểm nhận qua các tuyến có sức chứa.
- Ví dụ mạng máy tính: dữ liệu đi từ nguồn đến đích qua các liên kết có băng thông.
- Ví dụ phân công: sinh viên được gán vào nhóm thực hành hoặc đề tài có số lượng tối đa.
- Câu hỏi chung: có thể chuyển được bao nhiêu đơn vị từ nguồn đến đích mà không vượt dung lượng?

Từ bài toán thực tế đến mô hình

Bài toán thực tế

Có nhiều tuyến trung gian, mỗi tuyến có sức chứa hữu hạn.

- Hàng hóa: tuyến đường có tải trọng.
- Dữ liệu: liên kết có băng thông.
- Phân công: nhóm/lớp có sức chứa.

Mô hình dòng chảy

Tài nguyên đi từ nguồn s đến đích t qua các cạnh có dung lượng.

- Không vượt sức chứa.
- Không tự sinh hoặc mất đi tại đỉnh trung gian.
- Tối đa hóa tổng lượng đến đích.

Ba câu hỏi để nhận dạng bài toán max-flow

Một bài toán thường phù hợp với max-flow nếu trả lời được ba câu hỏi sau:

- ❶ **Nguồn và đích là gì?** Tài nguyên bắt đầu từ đâu và kết thúc ở đâu?
- ❷ **Dung lượng nằm ở đâu?** Cạnh, đỉnh, nhóm, tuyến hay liên kết nào có giới hạn?
- ❸ **Luồng có được bảo toàn không?** Tài nguyên đi qua đỉnh trung gian có bị biến đổi không?

Dấu hiệu quan trọng

Nếu bài toán chỉ cần tìm đường ngắn nhất hoặc đường rẻ nhất cho một đơn vị hàng hóa, đó thường không phải max-flow cơ bản.

Phân biệt max-flow với các bài toán gần giống

Bài toán	Câu hỏi chính	Mô hình phù hợp
Đường đi ngắn nhất	Đi từ s đến t rẻ nhất/nhanh nhất	Shortest path
Luồng cực đại	Đẩy được nhiều nhất từ s đến t	Max-flow
Luồng chi phí nhỏ nhất	Đẩy đủ lượng yêu cầu với chi phí thấp nhất	Min-cost flow
Ghép cặp	Chọn nhiều cặp không xung đột	Matching hoặc max-flow
Phân công có ưu tiên	Vừa xếp được người, vừa tối ưu nguyện vọng	Min-cost max-flow

Mạng dòng chảy

Một mạng dòng chảy là bộ $(G = (V, E), s, t, c)$, trong đó:

- G là đồ thị có hướng;
- $s \in V$ là nguồn, $t \in V$ là đích, $s \neq t$;
- $c : E \rightarrow \mathbb{R}_{\geq 0}$ là hàm dung lượng.

Quy ước: nếu $(u, v) \notin E$ thì $c(u, v) = 0$.

Luồng hợp lệ

Một luồng là hàm $f : V \times V \rightarrow \mathbb{R}$ thỏa mãn:

① **Ràng buộc dung lượng:**

$$0 \leq f(u, v) \leq c(u, v), \quad \forall (u, v) \in E.$$

② **Bảo toàn luồng:**

$$\sum_{u \in V} f(u, v) = \sum_{w \in V} f(v, w), \quad \forall v \in V \setminus \{s, t\}.$$

Giá trị luồng

$$\text{val}(f) = \sum_{v \in V} f(s, v) - \sum_{v \in V} f(v, s).$$

Nếu không có cạnh đi vào s , công thức rút gọn thành $\text{val}(f) = \sum_v f(s, v)$.

Bài toán

Tìm luồng hợp lệ f sao cho $\text{val}(f)$ là lớn nhất.

- Đầu vào: đồ thị có hướng, nguồn s , đích t , dung lượng các cạnh.
- Đầu ra: giá trị luồng cực đại và lượng luồng trên từng cạnh.
- Nghiệm cần kiểm tra hai điều kiện: không vượt dung lượng và bảo toàn tại mọi đỉnh trung gian.

Lưu ý mô hình hóa

Không phải mọi bài toán “đi từ A đến B” đều là max-flow. Cần có khái niệm dung lượng và bảo toàn luồng.

Lát cắt s - t

Định nghĩa

Một lát cắt s - t là phân hoạch (S, T) của V sao cho $s \in S$ và $t \in T$.

Dung lượng của lát cắt là:

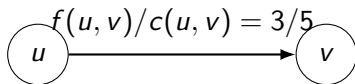
$$c(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

- Mọi luồng từ s đến t đều phải đi qua mọi lát cắt s - t .
- Vì vậy, với mọi luồng hợp lệ f và mọi lát cắt (S, T) :

$$\text{val}(f) \leq c(S, T).$$

- Lát cắt là một *chặn trên* cho giá trị luồng cực đại.

Đọc ký hiệu trên một cạnh



- $c(u, v) = 5$: cạnh có thể chở tối đa 5 đơn vị.
- $f(u, v) = 3$: hiện đang có 3 đơn vị đi qua cạnh.
- Dung lượng còn lại trên cạnh thuận là $5 - 3 = 2$.
- Có thể hoàn tác tối đa 3 đơn vị qua cạnh ngược trong mạng dư.

Kiểm tra bảo toàn luồng tại một đỉnh

Với đỉnh trung gian v , cần có:

tổng luồng vào v = tổng luồng ra khỏi v .

Hợp lệ

Vào: $2 + 3 = 5$; ra: $1 + 4 = 5$.

$$\sum_u f(u, v) = \sum_w f(v, w) = 5.$$

Không hợp lệ

Vào: 5; ra: 6.

$$\sum_u f(u, v) \neq \sum_w f(v, w).$$

Đỉnh trung gian không được tự sinh thêm luồng.

Ví dụ luồng hợp lệ và không hợp lệ

Xét một mạng có các cạnh ra khỏi s và qua hai đỉnh trung gian a, b .

Tiêu chí	Luồng A	Luồng B
Vượt dung lượng?	Không	Có cạnh $a \rightarrow t$ vượt dung lượng
Bảo toàn tại a ?	Có	Không
Bảo toàn tại b ?	Có	Có
Kết luận	Luồng hợp lệ	Không phải luồng hợp lệ

Nguyên tắc khi chấm bài

Không chỉ tính giá trị luồng. Trước hết phải kiểm tra nghiệm có hợp lệ hay không.

Tính dung lượng lát cắt bằng tay

Với lát cắt (S, T) , chỉ cộng dung lượng các cạnh **đi từ S sang T** .

- Không cộng cạnh nằm hoàn toàn trong S .
- Không cộng cạnh nằm hoàn toàn trong T .
- Không cộng cạnh đi từ T về S .

Ví dụ

Nếu $S = \{s, a\}$, $T = \{b, t\}$ và các cạnh từ S sang T là $s \rightarrow b$ dung lượng 2, $a \rightarrow b$ dung lượng 1, $a \rightarrow t$ dung lượng 2, thì

$$c(S, T) = 2 + 1 + 2 = 5.$$

Ý chính

Mạng dư mô tả những thao tác còn có thể thực hiện trên luồng hiện tại.

Với mỗi cạnh gốc (u, v) và luồng hiện tại f , ta có:

$$c_f(u, v) = c(u, v) - f(u, v) \quad (\text{cạnh thuận}),$$

$$c_f(v, u) = f(u, v) \quad (\text{cạnh ngược}).$$

- Cạnh thuận cho biết còn tăng được bao nhiêu trên cạnh (u, v) .
- Cạnh ngược cho biết có thể giảm/hoàn tác bao nhiêu luồng đã gửi qua (u, v) .
- Chỉ giữ các cạnh có dung lượng dư dương.

Đường tăng luồng

Định nghĩa

Đường tăng luồng là một đường từ s đến t trong mạng dư.

Nếu P là một đường tăng luồng, lượng tăng tối đa theo đường đó là nút cổ chai:

$$\Delta = \min_{(u,v) \in P} c_f(u, v).$$

- Với cạnh thuận, tăng $f(u, v)$ thêm Δ .
- Với cạnh ngược của cạnh gốc (v, u) , giảm $f(v, u)$ đi Δ .
- Sau cập nhật, luồng vẫn hợp lệ và giá trị luồng tăng thêm Δ .

Câu hỏi

Vì sao cần cạnh ngược nếu mục tiêu là tăng luồng?

- Một quyết định tăng luồng trước đó có thể “chiếm chỗ” của lựa chọn tốt hơn về sau.
- Cạnh ngược cho phép thuật toán điều chỉnh lại luồng mà không phá vỡ tính hợp lệ.
- Nếu bỏ cạnh ngược, thuật toán tham lam theo đường có thể mắc kẹt ở nghiệm chưa tối ưu.

Điểm cần nhớ

Mạng dư không chỉ biểu diễn khả năng tăng trực tiếp, mà còn biểu diễn khả năng tái phân phối luồng.

Mạng dư khi luồng ban đầu bằng 0

Nếu $f(u, v) = 0$ với mọi cạnh gốc (u, v) , thì:

$$c_f(u, v) = c(u, v), \quad c_f(v, u) = 0.$$

- Ban đầu mạng dư trùng với mạng gốc nếu bỏ các cạnh ngược dung lượng 0.
- Mọi đường từ s đến t trong mạng gốc đều là đường tăng luồng ban đầu.
- Sau lần tăng đầu tiên, các cạnh ngược mới bắt đầu xuất hiện với dung lượng dương.

Mạng dư sau một lần tăng luồng

Giả sử cạnh gốc $u \rightarrow v$ có dung lượng 5 và sau một bước ta gửi 3 đơn vị qua cạnh này.

$$f(u, v) = 3, \quad c(u, v) = 5.$$

Khi đó trong mạng dư có:

$$c_f(u, v) = 5 - 3 = 2, \quad c_f(v, u) = 3.$$

Diễn giải

Ta còn có thể tăng thêm 2 đơn vị theo chiều $u \rightarrow v$, hoặc giảm/hoàn tác tối đa 3 đơn vị theo chiều ngược $v \rightarrow u$.

Vì sao bỏ cạnh ngược có thể sai?

- Một đường tăng luồng được chọn sớm có thể dùng một cạnh trung gian quan trọng.
- Về sau, thuật toán có thể cần “rút bớt” luồng trên cạnh đó để mở đường cho một phương án tốt hơn.
- Cạnh ngược chính là thao tác rút bớt nhưng vẫn bảo toàn luồng toàn cục.

Sai lầm phổ biến

Coi mạng dư chỉ gồm các cạnh còn thừa dung lượng theo chiều gốc. Cách hiểu này làm mất khả năng điều chỉnh nghiệm.

Checklist xây dựng mạng dư

Với mỗi cạnh gốc (u, v) :

- 1 Tính dung lượng dư thuận $c(u, v) - f(u, v)$.
- 2 Nếu giá trị này dương, thêm cạnh dư $u \rightarrow v$.
- 3 Tính dung lượng dư ngược $f(u, v)$.
- 4 Nếu giá trị này dương, thêm cạnh dư $v \rightarrow u$.

Kiểm tra nhanh

Mỗi cạnh gốc sinh ra tối đa hai cạnh trong mạng dư: một cạnh thuận và một cạnh ngược.

Thuật toán tăng luồng

Bắt đầu với luồng bằng 0. Khi còn đường tăng luồng trong mạng dư, tăng luồng theo đường đó.

- Mỗi vòng lặp cần tìm một đường từ s đến t trong mạng dư.
- Lượng tăng là dung lượng dư nhỏ nhất trên đường vừa tìm.
- Dừng khi không còn đường tăng luồng.

Điều kiện kết thúc

Nếu tất cả dung lượng là số nguyên, Ford–Fulkerson kết thúc sau hữu hạn bước. Với dung lượng vô tỉ và cách chọn đường không phù hợp, thuật toán có thể không kết thúc.

Diễn giải từng vòng lặp Ford–Fulkerson

Mỗi vòng lặp gồm bốn thao tác:

- 1 Xây dựng hoặc cập nhật mạng dư từ luồng hiện tại.
- 2 Tìm một đường tăng luồng P từ s đến t .
- 3 Tính nút cổ chai Δ trên đường P .
- 4 Cập nhật luồng và các cạnh dư tương ứng.

Điều làm thuật toán đúng

Mỗi bước tăng làm giá trị luồng tăng lên, nhưng không làm vi phạm dung lượng hay bảo toàn luồng.

Pseudocode Ford–Fulkerson

```
1: Khởi tạo  $f(u, v) \leftarrow 0$  với mọi cạnh  $(u, v)$ 
2: while tồn tại đường tăng luồng  $P$  từ  $s$  đến  $t$  trong mạng dư do
3:    $\Delta \leftarrow \min\{c_f(u, v) : (u, v) \in P\}$ 
4:   for all cạnh  $(u, v)$  trên  $P$  do
5:     if  $(u, v)$  là cạnh thuận của mạng gốc then
6:        $f(u, v) \leftarrow f(u, v) + \Delta$ 
7:     else
8:        $f(v, u) \leftarrow f(v, u) - \Delta$ 
9:     end if
10:  end for
11: end while
12: Trả về  $f$ 
```

Bất biến

Sau mỗi lần cập nhật, f vẫn thỏa mãn ràng buộc dung lượng và bảo toàn luồng.

Mệnh đề

Nếu thuật toán dừng, luồng hiện tại là luồng cực đại.

- Khi thuật toán dừng, không còn đường từ s đến t trong mạng dư.
- Gọi S là tập các đỉnh còn đi tới được từ s trong mạng dư, và $T = V \setminus S$.
- Khi đó mọi cạnh từ S sang T trong mạng gốc đã bão hòa, còn mọi cạnh từ T sang S có luồng bằng 0.
- Suy ra $\text{val}(f) = c(S, T)$.
- Vì mọi luồng đều không vượt quá mọi lát cắt, f là cực đại và (S, T) là lát cắt nhỏ nhất.

Chứng chỉ dừng của Ford–Fulkerson

Khi không còn đường tăng luồng trong mạng dư:

- ❶ Lấy S là tập các đỉnh đi tới được từ s trong mạng dư.
- ❷ Đặt $T = V \setminus S$; khi đó $t \in T$.
- ❸ Mọi cạnh gốc từ S sang T đều đã bão hòa.
- ❹ Mọi cạnh gốc từ T sang S có luồng bằng 0.

Kết luận

Giá trị luồng bằng dung lượng lát cắt (S, T) , nên luồng là cực đại.

Edmonds–Karp: BFS tìm đường tăng luồng

Trong Edmonds–Karp, mỗi lần tìm đường tăng luồng dùng BFS trên mạng dư.

- Mỗi cạnh dư có dung lượng dương được xem như một cạnh có thể đi qua.
- BFS bảo đảm đường tìm được có số cạnh ít nhất.
- Mảng cha parent dùng để truy vết đường từ t về s .
- Sau khi tìm được đường, lần ngược để tính nút cổ chai.

Ý nghĩa thực hành

Edmonds–Karp dễ cài đặt, ổn định cho bài tập và dữ liệu vừa phải.

Ý tưởng

Edmonds–Karp là Ford–Fulkerson với quy tắc chọn đường tăng luồng bằng BFS trong mạng dư.

- BFS tìm đường có số cạnh ít nhất từ s đến t .
- Quy tắc này loại bỏ sự phụ thuộc vào cách chọn đường tùy ý.
- Số lần tăng luồng là $O(|V||E|)$.
- Mỗi lần BFS tốn $O(|E|)$.

Độ phức tạp

Edmonds–Karp chạy trong thời gian $O(|V||E|^2)$.

So sánh nhanh

Thuật toán	Cách chọn đường tăng luồng	Độ phức tạp thường nêu
Ford–Fulkerson	Bất kỳ đường nào trong mạng dư	$O(E \cdot \text{val}(f^*))$ nếu dung lượng nguyên
Edmonds–Karp	BFS, đường ít cạnh nhất	$O(V E ^2)$
Dinic	BFS tạo level graph, DFS gửi blocking flow	$O(V ^2 E)$ trong trường hợp tổng quát

Lưu ý

Chặn $O(|E| \cdot \text{val}(f^*))$ là giả đa thức, vì phụ thuộc vào giá trị luồng cực đại chứ không chỉ phụ thuộc vào số bit của đầu vào.

Dinic: ý tưởng cấp độ

Dinic cải thiện cách gửi luồng bằng cách tổ chức mạng dư theo từng lớp.

- BFS từ s tạo **level graph**: mỗi đỉnh có một mức khoảng cách từ s .
- Chỉ giữ các cạnh đi từ mức i sang mức $i + 1$.
- Trên level graph, thuật toán gửi một **blocking flow**.
- Sau đó xây lại level graph và lặp lại.

Trực giác

Thay vì tăng luồng theo từng đường đơn lẻ, Dinic cố gắng đẩy nhiều đường cùng một pha BFS.

Blocking flow là gì?

Một blocking flow trên level graph là một luồng làm cho mọi đường từ s đến t trong level graph bị chặn bởi ít nhất một cạnh bão hòa.

- Không nhất thiết đã là max-flow của mạng gốc.
- Nhưng nó loại bỏ toàn bộ các đường tăng luồng ngắn nhất hiện tại.
- Vì vậy khoảng cách lớp từ s đến t sẽ tăng trong pha sau.

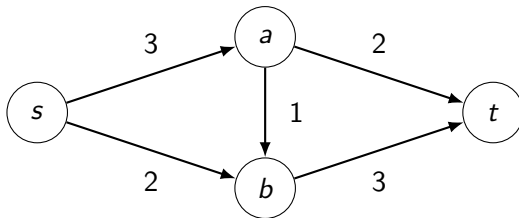
Mức độ bài học

Trong bài này, Dinic được giới thiệu để định hướng thực hành; trọng tâm cài đặt vẫn là Edmonds–Karp.

Khi nào nên dùng thuật toán nào?

Tình huống	Gợi ý
Học khái niệm, làm tay	Ford–Fulkerson
Cài đặt đầu tiên, dữ liệu nhỏ/vừa	Edmonds–Karp
Dữ liệu lớn hơn, nhiều cạnh	Dinic
Có chi phí trên cạnh	Min-cost max-flow
Có cận dưới bắt buộc	Flow with lower bounds

Ví dụ mạng nhỏ



Cạnh được ghi bằng dung lượng. Bắt đầu với luồng bằng 0 trên mọi cạnh.

Bước 1: chọn đường $s \rightarrow a \rightarrow t$

- Dung lượng dư trên $s \rightarrow a$ là 3.
- Dung lượng dư trên $a \rightarrow t$ là 2.
- Nút cổ chai:

$$\Delta = \min\{3, 2\} = 2.$$

- Cập nhật: $f(s, a) = 2, f(a, t) = 2$.
- Giá trị luồng tăng từ 0 lên 2.

Bước 2: chọn đường $s \rightarrow b \rightarrow t$

- Dung lượng dư trên $s \rightarrow b$ là 2.
- Dung lượng dư trên $b \rightarrow t$ là 3.
- Nút cổ chai:

$$\Delta = \min\{2, 3\} = 2.$$

- Cập nhật: $f(s, b) = 2$, $f(b, t) = 2$.
- Giá trị luồng tăng từ 2 lên 4.

Bước 3: chọn đường $s \rightarrow a \rightarrow b \rightarrow t$

Sau hai bước đầu, còn dung lượng dư:

$$c_f(s, a) = 1, \quad c_f(a, b) = 1, \quad c_f(b, t) = 1.$$

Vì vậy có đường tăng luồng $s \rightarrow a \rightarrow b \rightarrow t$ với

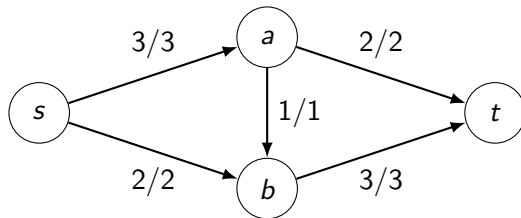
$$\Delta = \min\{1, 1, 1\} = 1.$$

Cập nhật:

$$f(s, a) = 3, \quad f(a, b) = 1, \quad f(b, t) = 3.$$

Giá trị luồng tăng từ 4 lên 5.

Luồng cuối cùng trên từng cạnh



Ký hiệu trên cạnh là f/c . Các cạnh ra khỏi nguồn đều bão hòa.

Mạng dư cuối và min-cut

Ở luồng cuối, từ s không còn cạnh dư đi ra vì:

$$c_f(s, a) = 3 - 3 = 0, \quad c_f(s, b) = 2 - 2 = 0.$$

Do đó tập đỉnh đi tới được từ s trong mạng dư là $S = \{s\}$.

$$T = \{a, b, t\}.$$

Dung lượng lát cắt:

$$c(S, T) = c(s, a) + c(s, b) = 3 + 2 = 5.$$

Vì giá trị luồng cũng bằng 5, đây là chứng chỉ tối ưu.

Ba lần tăng luồng

Bước	Đường tăng luồng	Nút cổ chai Δ	Giá trị luồng sau bước
1	$s \rightarrow a \rightarrow t$	2	2
2	$s \rightarrow b \rightarrow t$	2	4
3	$s \rightarrow a \rightarrow b \rightarrow t$	1	5

- Sau bước 3, hai cạnh ra khỏi nguồn đều bão hòa: $s \rightarrow a$ có luồng 3/3, $s \rightarrow b$ có luồng 2/2.
- Không còn đường tăng luồng từ s đến t trong mạng dư.
- Lát cắt $S = \{s\}$, $T = \{a, b, t\}$ có dung lượng $c(S, T) = 3 + 2 = 5$.

Kết luận

Giá trị max-flow là 5 và lát cắt trên là một min-cut.

Kiểm tra nghiệm bằng định nghĩa

Luồng cuối cùng:

$$f(s, a) = 3, \quad f(s, b) = 2, \quad f(a, b) = 1, \quad f(a, t) = 2, \quad f(b, t) = 3.$$

- Không cạnh nào vượt dung lượng.
- Tại a : vào 3, ra $1 + 2 = 3$.
- Tại b : vào $2 + 1 = 3$, ra 3.
- Giá trị luồng: $f(s, a) + f(s, b) = 5$.
- Min-cut có dung lượng 5, nên không thể có luồng lớn hơn 5.

Ghép cặp hai phía

Bài toán

Cho tập trái L , tập phải R và các cạnh tương thích giữa L và R . Tìm số cặp ghép lớn nhất sao cho mỗi đỉnh xuất hiện trong nhiều nhất một cặp.

- Tạo nguồn s và đích t .
- Thêm cạnh $s \rightarrow u$ với dung lượng 1 cho mọi $u \in L$.
- Thêm cạnh $u \rightarrow v$ với dung lượng 1 nếu $u \in L$ tương thích với $v \in R$.
- Thêm cạnh $v \rightarrow t$ với dung lượng 1 cho mọi $v \in R$.

Kết quả

Giá trị max-flow bằng kích thước ghép cặp lớn nhất.

Phân công sinh viên vào nhóm thực hành

Mô hình

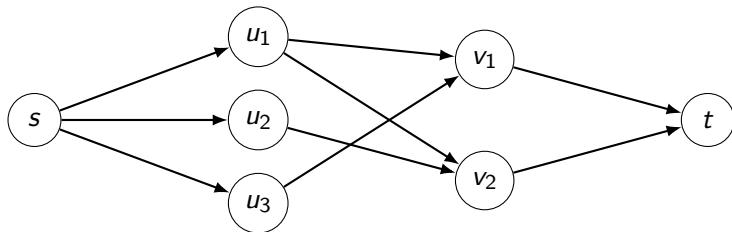
Mỗi sinh viên được chọn tối đa một nhóm; mỗi nhóm có sức chứa tối đa.

- Cạnh $s \rightarrow$ sinh viên: dung lượng 1.
- Cạnh sinh viên $\rightarrow$ nhóm: dung lượng 1 nếu sinh viên đăng ký/phù hợp nhóm đó.
- Cạnh nhóm $\rightarrow t$: dung lượng bằng sức chứa tối đa của nhóm.
- Nếu cần bắt buộc mỗi sinh viên được xếp nhóm, kiểm tra max-flow có bằng số sinh viên hay không.

Ràng buộc tối thiểu

Nếu nhóm cần số lượng tối thiểu, bài toán cần mở rộng bằng ràng buộc cận dưới hoặc xử lý kiểm tra bổ sung; mô hình max-flow cơ bản chỉ biểu diễn trực tiếp cận trên.

Mô hình hóa ghép cặp hai phía bằng hình dung



Tất cả cạnh có dung lượng 1. Mỗi đơn vị luồng tương ứng với một cặp được chọn.

Ràng buộc dung lượng trên đỉnh

Max-flow cơ bản giới hạn dung lượng trên cạnh. Nếu một đỉnh v chỉ được cho đi qua tối đa b_v đơn vị, ta tách đỉnh:

$$v \longrightarrow v_{in} \rightarrow v_{out}$$

với cạnh $v_{in} \rightarrow v_{out}$ có dung lượng b_v .

- Mọi cạnh đi vào v cũ được nối vào v_{in} .
- Mọi cạnh đi ra khỏi v cũ được nối từ v_{out} .
- Cạnh giữa v_{in} và v_{out} kiểm soát tổng lượng đi qua đỉnh.

Đường đi rời cạnh

Bài toán

Tìm số đường đi từ s đến t sao cho không có hai đường dùng chung một cạnh.

- Gán dung lượng 1 cho mọi cạnh.
- Mỗi đơn vị luồng tương ứng với một đường đi sử dụng các cạnh riêng.
- Giá trị max-flow bằng số đường đi rời cạnh lớn nhất.

Mở rộng

Nếu cần đường đi rời đỉnh, dùng kỹ thuật tách đỉnh với dung lượng đỉnh bằng 1.

Từ max-flow sang min-cost max-flow

Trong nhiều bài toán phân công, chỉ xếp được người vào nhóm là chưa đủ; ta còn muốn xếp theo mức ưu tiên.

- Cạnh sinh viên $\rightarrow$ nhóm có thể gán chi phí theo nguyện vọng.
- Mục tiêu: đẩy được đủ luồng và tổng chi phí nhỏ nhất.
- Khi đó bài toán chuyển thành min-cost max-flow.

Ví dụ

Nếu sinh viên chọn nguyện vọng 1, 2, 3, ta có thể đặt chi phí lần lượt là 0, 1, 2 để ưu tiên nguyện vọng cao.

Khi nào không nên dùng max-flow cơ bản?

Max-flow cơ bản chưa đủ khi bài toán có:

- Chi phí khác nhau trên các cạnh và cần tối ưu tổng chi phí: dùng min-cost flow.
- Nhu cầu/cận dưới bắt buộc trên cạnh hoặc tại nhóm: dùng flow with lower bounds.
- Nhiều loại tài nguyên đồng thời: có thể cần quy hoạch tuyến tính hoặc mô hình tối ưu tổ hợp khác.
- Mục tiêu phi tuyến hoặc phụ thuộc vào thứ tự: max-flow không còn là mô hình trực tiếp.

Nguyên tắc

Chọn mô hình theo ràng buộc và hàm mục tiêu, không chọn thuật toán trước rồi ép bài toán theo thuật toán.

Gợi ý cài đặt Edmonds–Karp

- Lưu danh sách kề của mạng dư.
- Với mỗi cạnh, lưu thêm chỉ số cạnh ngược để cập nhật nhanh.
- BFS lưu mảng parent để truy vết đường tăng luồng.
- Sau khi tìm được t , lần ngược để tính nút cổ chai Δ .
- Cập nhật đồng thời cạnh thuận và cạnh ngược trong cấu trúc dữ liệu.

Mẫu kiểm tra nhanh

Với ví dụ nhỏ ở trên, chương trình phải trả giá trị max-flow bằng 5.

Cấu trúc cạnh trong cài đặt C++

Một cách cài đặt phổ biến là lưu cạnh thuận và cạnh ngược theo cặp.

```
struct Edge {  
    int to, rev;  
    long long cap;  
};  
vector<vector<Edge>> adj;
```

- to: đỉnh kề.
- rev: vị trí của cạnh ngược trong danh sách kề của to.
- cap: dung lượng dư hiện tại.

Thêm cạnh và cạnh ngược

```
void addEdge(int u, int v, long long c) {  
    Edge a{v, (int)adj[v].size(), c};  
    Edge b{u, (int)adj[u].size(), 0};  
    adj[u].push_back(a);  
    adj[v].push_back(b);  
}
```

Ý nghĩa

Ban đầu cạnh ngược có dung lượng dư 0. Khi gửi luồng qua cạnh thuận, dung lượng cạnh thuận giảm và dung lượng cạnh ngược tăng.

Mẫu kiểm thử tối thiểu

Khi cài đặt, nên kiểm thử theo thứ tự:

- 1 Mạng không có đường từ s đến t : kết quả 0.
- 2 Một đường duy nhất: kết quả bằng dung lượng nhỏ nhất trên đường.
- 3 Hai đường song song độc lập: kết quả bằng tổng hai nút cổ chai.
- 4 Ví dụ trong bài: kết quả 5.
- 5 Một ví dụ cạnh ngược để đạt tối ưu.

Trường hợp biên cần kiểm thử

- Không có đường từ s đến t : kết quả phải là 0.
- Có cạnh dung lượng 0: cạnh đó không được dùng trong mạng dư.
- Nhiều đường tăng luồng cùng tồn tại: kết quả cuối vẫn phải giống nhau, dù đường chọn khác nhau.
- Đồ thị có chu trình: BFS/DFS không được lặp vô hạn.
- Dung lượng lớn: tránh kiểu dữ liệu số nguyên bị tràn.

Sai lầm thường gặp

- Chỉ cập nhật cạnh thuận mà quên cập nhật cạnh ngược.
- Nhầm dung lượng còn lại $c(u, v) - f(u, v)$ với dung lượng gốc $c(u, v)$.
- Không kiểm tra bảo toàn luồng tại đỉnh trung gian khi debug.
- Tính min-cut từ đồ thị gốc thay vì từ mạng dư sau khi thuật toán dừng.
- Áp dụng max-flow cho bài toán có chi phí mà không chuyển sang min-cost flow.

- Max-flow tìm lượng luồng lớn nhất từ nguồn đến đích dưới ràng buộc dung lượng và bảo toàn.
- Mạng dư là công cụ trung tâm để mô tả khả năng tăng và điều chỉnh luồng.
- Ford–Fulkerson tăng luồng theo đường tăng luồng; Edmonds–Karp dùng BFS để có chặn đa thức.
- Định lý max-flow min-cut cho chứng chỉ tối ưu: giá trị luồng cực đại bằng dung lượng lát cắt nhỏ nhất.
- Ghép cặp hai phía và phân công có sức chứa là các ứng dụng điển hình.

Câu hỏi nhanh 1

- 1 Nếu một cạnh gốc có $c(u, v) = 10$ và $f(u, v) = 4$, dung lượng dư thuận và ngược là bao nhiêu?
- 2 Nếu không còn đường từ s đến t trong mạng dư, có thể kết luận gì?
- 3 Vì sao lát cắt cho ta chặn trên của mọi luồng?

Đáp án gợi ý

$c_f(u, v) = 6$, $c_f(v, u) = 4$; luồng hiện tại là cực đại; mọi luồng từ s đến t phải đi qua lát cắt.

Câu hỏi nhanh 2

Đánh dấu đúng/sai:

- ① Edmonds–Karp là Ford–Fulkerson với BFS.
- ② Cạnh ngược trong mạng dư là cạnh có sẵn trong đồ thị gốc.
- ③ Nếu một luồng có giá trị bằng dung lượng của một lát cắt, luồng đó là cực đại.
- ④ Max-flow cơ bản có thể tối ưu chi phí phân công nếu mỗi cạnh có chi phí khác nhau.

Đáp án

Đúng; Sai; Đúng; Sai.

- 1 Với mạng ví dụ trong bài, hãy thử chọn đường tăng luồng theo thứ tự khác và kiểm tra giá trị cuối.
- 2 Tự xây dựng một mạng có ít nhất 5 đỉnh, tính max-flow và chỉ ra một min-cut tương ứng.
- 3 Mô hình hóa bài toán xếp sinh viên vào nhóm thực hành, trong đó mỗi nhóm có sức chứa khác nhau.
- 4 Viết pseudocode BFS để tìm đường tăng luồng trong Edmonds–Karp.
- 5 Chỉ ra một tình huống bài toán phân công cần min-cost flow thay vì max-flow cơ bản.

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein. *Introduction to Algorithms*, chương về Maximum Flow.
- J. Kleinberg, É. Tardos. *Algorithm Design*, chương về Network Flow.
- MIT 6.046J Design and Analysis of Algorithms, lectures on maximum flow.
- NetworkX documentation: flow algorithms, dùng để đối chiếu cài đặt và thử nghiệm nhỏ.